Dossier de prompts — Nouveautés (à exécuter en dernier)

Prompts pour ChatGPT · version 1.0 · 31 octobre 2026 · travail sur main, sans branche, PR, CI ni déploiement sans accord explicite

À quoi sert ce dossier

Fonctions novatrices choisies avec vous, à exécuter **uniquement après tous les autres prompts**, après la refonte du design (série 8) et, idéalement, après le module Caméras (série 9).

Contenu

Réf.	Fonction	Application
10.1 à 10.4	Commande par QR de table, paiement confirmé par SMS avec reçu téléchargeable, stock automatique, compte à rebours de service	Zipple
10.5	Prévenir le client par WhatsApp quand son service est prêt	NAXA Services
10.6	QR « Payer ici »	Bootik, Zipple, NAXA Services, Loyable, NAXA Hotel, CotiZ
10.7	Import d'un cahier papier par photo	Bootik, Zipple, NAXA Services, Loyable, CotiZ
10.8	Attestation d'activité vérifiable	Kiosko, Bootik, Zipple, NAXA Services, Loyable, NAXA Hotel
10.9	Assistant vocal en langue locale (étude puis prototype)	CotiZ
10.10	Recette des nouveautés	Toutes

Informations à me demander avant de lancer

- Les codes USSD exacts de paiement marchand d'Orange Money et de MTN MoMo : ne jamais les inventer.
- Le choix du prestataire WhatsApp Business payant, si vous voulez un jour l'envoi automatique.

Mode d'emploi

1. Ouvrez une conversation ChatGPT neuve pour chaque prompt (ou continuez la même conversation si elle reste courte). Collez le prompt en entier.
2. Dans le prompt, remplacez « [COLLE ICI LE FICHIER DE PASSATION] » par le fichier de passation produit à la fin de l'étape précédente. Pour le tout premier prompt (0.1), il n'y en a pas.
3. Un seul prompt à la fois. Ne passez au suivant qu'après avoir relu le résultat, fait les contrôles locaux indiqués et donné votre accord par écrit.
4. À chaque réponse, ChatGPT doit finir par un fichier de passation et par la phrase « Étape terminée. J'attends ton accord… ». Conservez ce fichier : c'est la mémoire du projet.

Règles de travail à respecter à chaque étape

1. Travaille uniquement sur la branche main. Ne crée aucune branche parallèle.
2. N'ouvre aucune pull request (PR).
3. Ne lance, ne déclenche et n'attends aucune vérification CI (aucun workflow GitHub Actions). Les tests se font en local uniquement, et seulement ceux que ce prompt demande.
4. Ne fais aucun déploiement : ni publication d'APK ou de release, ni migration appliquée à Supabase, ni fonction serveur déployée, ni mise en ligne d'une page. Prépare seulement les fichiers.
5. Tout ce qui précède est interdit sans mon accord explicite, écrit dans cette conversation, pour chaque action concernée. Si une étape semble l'exiger (y compris un push qui déclencherait un workflow automatique), arrête-toi, explique pourquoi et demande-moi.
6. Si tu n'as pas accès à mon dépôt, livre les fichiers complets avec leur chemin exact, prêts à copier ; ne suppose pas qu'ils existent déjà.

7. N'invente pas : si une information manque, pose-moi la question (une seule à la fois) avant de continuer.

Contrôles après chaque étape

- Le travail est bien sur main, sans branche ni pull request créées.
- Aucun workflow CI lancé, aucun déploiement, aucune migration appliquée sans votre accord.
- Les textes sont dans les fichiers FR/EN ; aucun PIN ni OTP n'apparaît dans le code, les journaux ou les messages.
- Les tests demandés par le prompt passent en local ; les calculs d'argent sont exacts au franc près.
- Le fichier de passation est complet (7 rubriques) et sauvegardé.

Les prompts

Chaque prompt ci-dessous est autonome : copiez-le en entier dans ChatGPT, en remplaçant [COLLE ICI LE FICHIER DE PASSATION] par le fichier de l'étape précédente.

PROMPT 10.1 — Zipple : commande par QR de table

RÔLE : Tu es un ingénieur logiciel senior (Android, Supabase et PostgreSQL, sécurité, interfaces mobiles pour téléphones d'entrée de gamme). Je suis porteur de projet et non développeur : explique brièvement, puis livre du code complet et prêt à l'emploi.

PROJET : Fonctionnalités novatrices à ajouter à la famille d'applications, à exécuter UNIQUEMENT après tous les autres prompts, après la refonte du design (série 8) et idéalement après le module Caméras (série 9). Contenu : (1) Zipple : commande par QR de table avec paiement mobile money ou espèces, validation par la serveuse, compte à rebours de service, confirmation « servi », et stock mis à jour automatiquement ; (2) NAXA Services : prévenir le client par WhatsApp quand son service est prêt ; (3) QR « Payer ici » dans les applications qui encaissent ; (4) import d'un cahier papier par photo ; (5) attestation d'activité vérifiable ; (6) CotiZ : étude puis prototype d'un assistant vocal en langue locale. Les applications existent déjà : réutilise leurs modules (lecture des SMS, stock, recettes, notifications, thème) au lieu de les réécrire.

ÉTAT DE DÉPART : voici le fichier de passation de l'étape précédente : [COLLE ICI LE FICHIER DE PASSATION]. S'il est vide ou absent, dis-le-moi avant de commencer.

MISSION : Permettre au client de commander depuis son téléphone en scannant le QR de sa table.

À FAIRE :

1. Chaque table a un QR (adresse du restaurant + numéro de table) et un petit code de table de 2 chiffres affiché sur la table, que le client saisit pour éviter les fausses commandes à distance. Page web du client : carte par catégories avec photos compressées, plats épuisés grisés, panier, total en francs CFA, choix du paiement.
2. Le client choisit : espèces, Orange Money ou MTN MoMo. Espèces : la commande passe « à encaisser » et la serveuse encaisse à la table. Mobile money : la page affiche le montant exact et une référence de commande, propose le lien de paiement (demande-moi les codes USSD exacts) et un champ facultatif pour l'identifiant de transaction.
3. Fonction serveur « créer une commande » (non déployée) : vérifie le code de table, la disponibilité, RECALCULE les prix côté serveur, applique la limite de commandes, renvoie un jeton de suivi ; le client ne lit que sa commande avec ce jeton.
4. Côté serveuse (application Zipple) : la commande arrive en temps réel avec notification sonore ; elle voit la table, les plats, le paiement et peut valider, modifier (retirer un plat épuisé, avec message au client) ou refuser avec raison. La serveuse met la carte à jour à tout moment (disponible, épuisé, prix du jour) et la page du client suit.
5. Statuts : envoyée, validée, en préparation, servie, pas servie, annulée, refusée ; la page du client affiche l'état en direct (mise à jour toutes les 5 secondes au plus, ou temps réel) et reste lisible si le réseau coupe un instant.

CONTRÔLES (en local uniquement, aucune CI) : Test local : commande de bout en bout avec un faux client ; prix falsifié côté client refusé ; fausse table refusée.

LIVRABLES : Code complet avec chemins exacts, migrations SQL ordonnées (non appliquées), textes FR/EN, tests locaux demandés, et le fichier de passation.

CONTRAINTES DU PROJET :

- Android uniquement. Hors connexion d'abord : tout fonctionne sur la base locale, la synchronisation se fait au retour du réseau.
- Uniquement des solutions gratuites (Supabase offre gratuite partagée, Firebase Cloud Messaging, bibliothèques libres, GitHub). Aucune dépense sans mon accord.
- Sécurité : aucun code PIN ni OTP n'est lu, stocké, transmis ni affiché ; un SMS qui en contient est ignoré. Aucun message WhatsApp ne contient de PIN ou d'OTP.
- L'IA propose, l'humain décide : jamais de validation de paiement ni de modification de solde par l'IA seule ; chaque suggestion explique pourquoi ; aucun numéro ni SMS brut envoyé à un service externe.
- Aucun texte en dur dans le code : tout est dans des fichiers de langue français et English. Montants en francs CFA, formatés (45 000 F), calculs exacts au franc près.
- Les accès clients sont créés uniquement par l'administration ; pas d'inscription libre dans l'application. Un seul backend Supabase partagé par les 7 applications, avec règles RLS strictes.
- L'application suit la charte graphique de son logo (couleurs, police, thème) ; états toujours doublés d'une icône et d'un libellé ; contraste texte d'au moins 4,5 : 1.

- Les pages vues par les clients finaux (menu par QR, suivi de service, paiement) sont des pages web légères : aucune installation, moins de 150 Ko au premier chargement, utilisables sur réseau lent, aux couleurs de l'application, en français et en English.
- Ne jamais faire confiance au téléphone du client : prix, totaux, disponibilité et statuts sont recalculés côté serveur ; limitation du nombre de commandes par appareil et par table ; code de table à saisir ; un client ne voit que sa propre commande grâce à un jeton aléatoire.
- Aucune donnée personnelle obligatoire pour le client final (le numéro de téléphone est facultatif et accompagné d'un consentement clair) ; aucune donnée sensible dans les liens.
- Pas d'envoi automatique payant : WhatsApp reste en message préparé envoyé en un clic, avec une couche d'envoi remplaçable (un prestataire payant pourra être branché plus tard sans réécrire l'application). Les notifications gratuites passent par la page web du client (mise à jour en direct et notification web si le client l'accepte).
- Ne jamais inventer un code USSD de paiement : demande-moi les codes exacts d'Orange Money et de MTN MoMo pour mon restaurant et mes commerces avant de coder les liens de paiement.
- Tout calcul de stock est atomique et idempotent : une même commande ne peut ni retirer deux fois le stock, ni le restituer deux fois ; jamais de stock négatif silencieux.

RÈGLES DE TRAVAIL OBLIGATOIRES :

1. Travaille uniquement sur la branche **main**. Ne crée aucune branche parallèle.
2. N'ouvre aucune pull request (PR).
3. Ne lance, ne déclenche et n'attends aucune vérification CI (aucun workflow GitHub Actions). Les tests se font en local uniquement, et seulement ceux que ce prompt demande.
4. Ne fais aucun déploiement : ni publication d'APK ou de release, ni migration appliquée à Supabase, ni fonction serveur déployée, ni mise en ligne d'une page. Prépare seulement les fichiers.
5. Tout ce qui précède est interdit **sans mon accord explicite, écrit dans cette conversation, pour chaque action concernée**. Si une étape semble l'exiger (y compris un push qui déclencherait un workflow automatique), arrête-toi, explique pourquoi et demande-moi.
6. Si tu n'as pas accès à mon dépôt, livre les fichiers complets avec leur chemin exact, prêts à copier ; ne suppose pas qu'ils existent déjà.
7. N'invente pas : si une information manque, pose-moi la question (une seule à la fois) avant de continuer.

MODÈLE DU FICHIER DE PASSATION À PRODUIRE EN FIN DE RÉPONSE :

FICHIER DE PASSATION — [application] — [numéro du prompt] — [date]

1. Fait : ce qui est terminé, en une ligne chacun.
2. Fichiers créés ou modifiés : chemins exacts.
3. Tables, fonctions et écrans créés : noms exacts.
4. Décisions techniques retenues et leur raison.
5. Points ouverts et questions pour moi.
6. Prochain prompt recommandé (numéro et titre).
7. Actions qui demandent mon accord explicite (push, migration, déploiement…).

Termine par : « Étape terminée. J'attends ton accord avant de passer à la suivante. »

PROMPT 10.2 — Zipple : paiement confirmé par SMS, notification et reçu téléchargeable

RÔLE : Tu es un ingénieur logiciel senior (Android, Supabase et PostgreSQL, sécurité, interfaces mobiles pour téléphones d'entrée de gamme). Je suis porteur de projet et non développeur : explique brièvement, puis livre du code complet et prêt à l'emploi.

PROJET : Fonctionnalités novatrices à ajouter à la famille d'applications, à exécuter UNIQUEMENT après tous les autres prompts, après la refonte du design (série 8) et idéalement après le module Caméras (série 9). Contenu : (1) Zipple : commande par QR de table avec paiement mobile money ou espèces, validation par la serveuse, compte à rebours de service, confirmation « servi », et stock mis à jour automatiquement ; (2) NAXA Services : prévenir le client par WhatsApp quand son service est prêt ; (3) QR « Payer ici » dans les applications qui encaissent ; (4) import d'un cahier papier par photo ; (5) attestation d'activité vérifiable ; (6) CotiZ : étude puis prototype d'un assistant vocal en langue locale. Les applications existent déjà : réutilise leurs modules (lecture des SMS, stock, recettes, notifications, thème) au lieu de les réécrire.

ÉTAT DE DÉPART : voici le fichier de passation de l'étape précédente : [COLLE ICI LE FICHIER DE PASSATION]. S'il est vide ou absent, dis-le-moi avant de commencer.

MISSION : Confirmer automatiquement le paiement mobile money et le dire au client.

À FAIRE :

1. Le téléphone de la serveuse (SIM des comptes marchands, permission SMS) lit le SMS de paiement avec la chaîne de lecture existante (OTP et PIN ignorés) et le rapproche de la commande : montant exact, horaire après l'envoi de la commande, référence ou identifiant de transaction si présent, numéro du payeur si connu.
2. Si un seul rapprochement est possible : la commande passe « payée » et le client voit « Paiement reçu » avec un reçu sur sa page (notification web si accordée, vibration et son). Si plusieurs commandes de même montant sont candidates ou en cas de doute : la serveuse choisit, jamais de rapprochement automatique ambigu.
3. Paiement non retrouvé après un délai réglable : la page du client propose de saisir l'identifiant de transaction ou d'appeler la serveuse ; la serveuse garde la décision finale de servir ou non.
4. Un SMS partagé ou saisi à la main ne confirme jamais seul un paiement sans passer par le contrôle de cohérence ; journal de chaque rapprochement ; une même transaction ne paie qu'une commande.
5. REÇU TÉLÉCHARGEABLE (bar et restaurant) : dès que la commande est payée (mobile money confirmé ou espèces encaissées par la serveuse), la page du client affiche « Télécharger mon reçu ». Contenu : nom et coordonnées de l'établissement (et numéro d'identification fiscale si renseigné), numéro de reçu à numérotation continue, numéro de commande, table, date et heure, articles avec quantités, prix unitaires et total, TVA si activée (taux paramétrable, à me faire confirmer), mode de paiement, référence de transaction, code QR de vérification d'authenticité, mention de remerciement ; en français ou en English selon la langue du client.
6. Formats et partage : image du reçu générée dans la page (sans bibliothèque lourde, pour respecter les 150 Ko), impression ou « Enregistrer en PDF » du navigateur, et bouton « Partager » (partage du téléphone, WhatsApp compris) ; version PDF officielle générée côté serveur (fonction non déployée) pour l'archive. Le lien du reçu reste valable un nombre de jours réglable (30 par défaut) et se retélécharge à volonté grâce au jeton de la commande.
7. Côté serveuse et manager : réimprimer ou renvoyer un reçu (lien WhatsApp préparé) ; l'addition imprimée d'une table sans commande par QR porte aussi un QR vers son reçu ; un reçu annulé ou corrigé produit un avoir numéroté et ne s'efface jamais ; export des reçus du jour, de la semaine ou du mois en PDF et Excel pour la comptabilité, avec nom de l'établissement et date en pied de page.

CONTRÔLES (en local uniquement, aucune CI) : Tests : paiement exact reconnu ; deux commandes de même montant = choix manuel ; SMS avec OTP ignoré ; même identifiant utilisé deux fois refusé ; reçu téléchargeable en image et en PDF, numérotation continue, avoir en cas d'annulation.

LIVRABLES : Code complet avec chemins exacts, migrations SQL ordonnées (non appliquées), textes FR/EN, tests locaux demandés, et le fichier de passation.

CONTRAINTES DU PROJET :

- Android uniquement. Hors connexion d'abord : tout fonctionne sur la base locale, la synchronisation se fait au retour du réseau.
- Uniquement des solutions gratuites (Supabase offre gratuite partagée, Firebase Cloud Messaging, bibliothèques libres, GitHub). Aucune dépense sans mon accord.
- Sécurité : aucun code PIN ni OTP n'est lu, stocké, transmis ni affiché ; un SMS qui en contient est ignoré. Aucun message WhatsApp ne contient de PIN ou d'OTP.
- L'IA propose, l'humain décide : jamais de validation de paiement ni de modification de solde par l'IA seule ; chaque suggestion explique pourquoi ; aucun numéro ni SMS brut envoyé à un service externe.
- Aucun texte en dur dans le code : tout est dans des fichiers de langue français et English. Montants en francs CFA, formatés (45 000 F), calculs exacts au franc près.
- Les accès clients sont créés uniquement par l'administration ; pas d'inscription libre dans l'application. Un seul backend Supabase partagé par les 7 applications, avec règles RLS strictes.
- L'application suit la charte graphique de son logo (couleurs, police, thème) ; états toujours doublés d'une icône et d'un libellé ; contraste texte d'au moins 4,5 : 1.
- Les pages vues par les clients finaux (menu par QR, suivi de service, paiement) sont des pages web légères : aucune installation, moins de 150 Ko au premier chargement, utilisables sur réseau lent, aux couleurs de l'application, en français et en English.
- Ne jamais faire confiance au téléphone du client : prix, totaux, disponibilité et statuts sont recalculés côté serveur ; limitation du nombre de commandes par appareil et par table ; code de table à saisir ; un client ne voit que sa propre commande grâce à un jeton aléatoire.
- Aucune donnée personnelle obligatoire pour le client final (le numéro de téléphone est facultatif et accompagné d'un consentement clair) ; aucune donnée sensible dans les liens.
- Pas d'envoi automatique payant : WhatsApp reste en message préparé envoyé en un clic, avec une couche d'envoi remplaçable (un prestataire payant pourra être branché plus tard sans réécrire l'application). Les notifications gratuites passent par la page web du client (mise à jour en direct et notification web si le client l'accepte).

- Ne jamais inventer un code USSD de paiement : demande-moi les codes exacts d'Orange Money et de MTN MoMo pour mon restaurant et mes commerces avant de coder les liens de paiement.
- Tout calcul de stock est atomique et idempotent : une même commande ne peut ni retirer deux fois le stock, ni le restituer deux fois ; jamais de stock négatif silencieux.

RÈGLES DE TRAVAIL OBLIGATOIRES :

1. Travaille uniquement sur la branche **main**. Ne crée aucune branche parallèle.
2. N'ouvre aucune pull request (PR).
3. Ne lance, ne déclenche et n'attends aucune vérification CI (aucun workflow GitHub Actions). Les tests se font en local uniquement, et seulement ceux que ce prompt demande.
4. Ne fais aucun déploiement : ni publication d'APK ou de release, ni migration appliquée à Supabase, ni fonction serveur déployée, ni mise en ligne d'une page. Prépare seulement les fichiers.
5. Tout ce qui précède est interdit **sans mon accord explicite, écrit dans cette conversation, pour chaque action concernée**. Si une étape semble l'exiger (y compris un push qui déclencherait un workflow automatique), arrête-toi, explique pourquoi et demande-moi.
6. Si tu n'as pas accès à mon dépôt, livre les fichiers complets avec leur chemin exact, prêts à copier ; ne suppose pas qu'ils existent déjà.
7. N'invente pas : si une information manque, pose-moi la question (une seule à la fois) avant de continuer.

MODÈLE DU FICHIER DE PASSATION À PRODUIRE EN FIN DE RÉPONSE :

FICHIER DE PASSATION — [application] — [numéro du prompt] — [date]

1. Fait : ce qui est terminé, en une ligne chacun.
2. Fichiers créés ou modifiés : chemins exacts.
3. Tables, fonctions et écrans créés : noms exacts.
4. Décisions techniques retenues et leur raison.
5. Points ouverts et questions pour moi.
6. Prochain prompt recommandé (numéro et titre).
7. Actions qui demandent mon accord explicite (push, migration, déploiement…).

Termine par : « Étape terminée. J'attends ton accord avant de passer à la suivante. »

PROMPT 10.3 — Zipple : stock automatique lié aux commandes

RÔLE : Tu es un ingénieur logiciel senior (Android, Supabase et PostgreSQL, sécurité, interfaces mobiles pour téléphones d'entrée de gamme). Je suis porteur de projet et non développeur : explique brièvement, puis livre du code complet et prêt à l'emploi.

PROJET : Fonctionnalités novatrices à ajouter à la famille d'applications, à exécuter UNIQUEMENT après tous les autres prompts, après la refonte du design (série 8) et idéalement après le module Caméras (série 9). Contenu : (1) Zipple : commande par QR de table avec paiement mobile money ou espèces, validation par la serveuse, compte à rebours de service, confirmation « servi », et stock mis à jour automatiquement ; (2) NAXA Services : prévenir le client par WhatsApp quand son service est prêt ; (3) QR « Payer ici » dans les applications qui encaissent ; (4) import d'un cahier papier par photo ; (5) attestation d'activité vérifiable ; (6) CotiZ : étude puis prototype d'un assistant vocal en langue locale. Les applications existent déjà : réutilise leurs modules (lecture des SMS, stock, recettes, notifications, thème) au lieu de les réécrire.

ÉTAT DE DÉPART : voici le fichier de passation de l'étape précédente : [COLLE ICI LE FICHIER DE PASSATION]. S'il est vide ou absent, dis-le-moi avant de commencer.

MISSION : Faire varier le stock automatiquement avec les commandes et restaurer en cas d'annulation.

À FAIRE :

1. Trois modes par article de la carte : illimité, quantité comptée (par exemple bouteilles) et recette (consommation d'ingrédients, en réutilisant la logique de recettes déjà construite). Choisis le mode au niveau de l'article.
2. Règle : à l'envoi de la commande le stock est RÉSERVÉ ; à la validation par la serveuse il est CONSOMMÉ ; une annulation ou un refus par la serveuse, ou l'expiration d'une commande non validée, RESTITUE le stock réservé ou consommé. Une annulation après préparation d'un plat enregistre une perte avec raison au lieu de restituer (la serveuse choisit : restituer ou perte).
3. Atomique et idempotent : mise à jour conditionnelle du stock (jamais en dessous de zéro), contrainte d'unicité par commande, article et type de mouvement ; journal de chaque mouvement (qui, quand, avant, après) ; deux clients commandant le dernier article en même temps : un seul gagne, l'autre voit « épuisé ».
4. Un article qui atteint zéro passe automatiquement « épuisé » sur la carte et prévient la serveuse ; alerte de seuil bas ; fonctionne hors connexion côté serveuse avec synchronisation sans doublon.
5. Les ventes sur place (sans QR) utilisent les mêmes règles de stock ; aucun double décompte entre commande QR et ticket de table.

CONTRÔLES (en local uniquement, aucune CI) : Tests : commande puis annulation = stock identique ; validation puis annulation avant préparation = restitué ; après préparation = perte ; dernier article disputé par deux clients.

LIVRABLES : Code complet avec chemins exacts, migrations SQL ordonnées (non appliquées), textes FR/EN, tests locaux demandés, et le fichier de passation.

CONTRAINTES DU PROJET :

- Android uniquement. Hors connexion d'abord : tout fonctionne sur la base locale, la synchronisation se fait au retour du réseau.
- Uniquement des solutions gratuites (Supabase offre gratuite partagée, Firebase Cloud Messaging, bibliothèques libres, GitHub). Aucune dépense sans mon accord.
- Sécurité : aucun code PIN ni OTP n'est lu, stocké, transmis ni affiché ; un SMS qui en contient est ignoré. Aucun message WhatsApp ne contient de PIN ou d'OTP.
- L'IA propose, l'humain décide : jamais de validation de paiement ni de modification de solde par l'IA seule ; chaque suggestion explique pourquoi ; aucun numéro ni SMS brut envoyé à un service externe.
- Aucun texte en dur dans le code : tout est dans des fichiers de langue français et English. Montants en francs CFA, formatés (45 000 F), calculs exacts au franc près.
- Les accès clients sont créés uniquement par l'administration ; pas d'inscription libre dans l'application. Un seul backend Supabase partagé par les 7 applications, avec règles RLS strictes.
- L'application suit la charte graphique de son logo (couleurs, police, thème) ; états toujours doublés d'une icône et d'un libellé ; contraste texte d'au moins 4,5 : 1.
- Les pages vues par les clients finaux (menu par QR, suivi de service, paiement) sont des pages web légères : aucune installation, moins de 150 Ko au premier chargement, utilisables sur réseau lent, aux couleurs de l'application, en français et en English.
- Ne jamais faire confiance au téléphone du client : prix, totaux, disponibilité et statuts sont recalculés côté serveur ; limitation du nombre de commandes par appareil et par table ; code de table à saisir ; un client ne voit que sa propre commande grâce à un jeton aléatoire.
- Aucune donnée personnelle obligatoire pour le client final (le numéro de téléphone est facultatif et accompagné d'un consentement clair) ; aucune donnée sensible dans les liens.
- Pas d'envoi automatique payant : WhatsApp reste en message préparé envoyé en un clic, avec une couche d'envoi remplaçable (un prestataire payant pourra être branché plus tard sans réécrire l'application). Les notifications gratuites passent par la page web du client (mise à jour en direct et notification web si le client l'accepte).
- Ne jamais inventer un code USSD de paiement : demande-moi les codes exacts d'Orange Money et de MTN MoMo pour mon restaurant et mes commerces avant de coder les liens de paiement.
- Tout calcul de stock est atomique et idempotent : une même commande ne peut ni retirer deux fois le stock, ni le restituer deux fois ; jamais de stock négatif silencieux.

RÈGLES DE TRAVAIL OBLIGATOIRES :

1. Travaille uniquement sur la branche **main**. Ne crée aucune branche parallèle.
2. N'ouvre aucune pull request (PR).
3. Ne lance, ne déclenche et n'attends aucune vérification CI (aucun workflow GitHub Actions). Les tests se font en local uniquement, et seulement ceux que ce prompt demande.
4. Ne fais aucun déploiement : ni publication d'APK ou de release, ni migration appliquée à Supabase, ni fonction serveur déployée, ni mise en ligne d'une page. Prépare seulement les fichiers.
5. Tout ce qui précède est interdit **sans mon accord explicite, écrit dans cette conversation, pour chaque action concernée**. Si une étape semble l'exiger (y compris un push qui déclencherait un workflow automatique), arrête-toi, explique pourquoi et demande-moi.
6. Si tu n'as pas accès à mon dépôt, livre les fichiers complets avec leur chemin exact, prêts à copier ; ne suppose pas qu'ils existent déjà.
7. N'invente pas : si une information manque, pose-moi la question (une seule à la fois) avant de continuer.

MODÈLE DU FICHIER DE PASSATION À PRODUIRE EN FIN DE RÉPONSE :

FICHIER DE PASSATION — [application] — [numéro du prompt] — [date]

1. Fait : ce qui est terminé, en une ligne chacun.
2. Fichiers créés ou modifiés : chemins exacts.
3. Tables, fonctions et écrans créés : noms exacts.
4. Décisions techniques retenues et leur raison.
5. Points ouverts et questions pour moi.
6. Prochain prompt recommandé (numéro et titre).
7. Actions qui demandent mon accord explicite (push, migration, déploiement…).

Termine par : « Étape terminée. J'attends ton accord avant de passer à la suivante. »

PROMPT 10.4 — Zipple : compte à rebours de service et confirmation « servi »

RÔLE : Tu es un ingénieur logiciel senior (Android, Supabase et PostgreSQL, sécurité, interfaces mobiles pour téléphones d'entrée de gamme). Je suis porteur de projet et non développeur : explique brièvement, puis livre du code complet et prêt à l'emploi.

PROJET : Fonctionnalités novatrices à ajouter à la famille d'applications, à exécuter UNIQUEMENT après tous les autres prompts, après la refonte du design (série 8) et idéalement après le module Caméras (série 9). Contenu : (1) Zipple : commande par QR de table avec paiement mobile money ou espèces, validation par la serveuse, compte à rebours de service, confirmation « servi », et stock mis à jour automatiquement ; (2) NAXA Services : prévenir le client par WhatsApp quand son service est prêt ; (3) QR « Payer ici » dans les applications qui encaissent ; (4) import d'un cahier papier par photo ; (5) attestation d'activité vérifiable ; (6) CotiZ : étude puis prototype d'un assistant vocal en langue locale. Les applications existent déjà : réutilise leurs modules (lecture des SMS, stock, recettes, notifications, thème) au lieu de les réécrire.

ÉTAT DE DÉPART : voici le fichier de passation de l'étape précédente : [COLLE ICI LE FICHIER DE PASSATION]. S'il est vide ou absent, dis-le-moi avant de commencer.

MISSION : Mesurer le temps d'attente et traiter les retards.

À FAIRE :

1. Réglage par restaurant : durée du compte à rebours (10 minutes par défaut, entre 1 et 60), moment de départ (à la validation par la serveuse par défaut, ou à la confirmation du paiement). L'échéance est enregistrée côté serveur et sert de référence, pas l'horloge du téléphone du client.
2. La page du client affiche le compte à rebours en direct et un bouton « J'ai été servi ». À l'échéance, elle propose « Pas encore servi ». Côté serveuse : commandes dépassées en rouge avec icône, tri par urgence.
3. Si le client signale « pas encore servi » : alerte au manager ; compensation configurable (aucune, boisson offerte, remise en pourcentage) appliquée avec validation du manager ; statistiques de temps de service moyen par plat, par serveuse et par heure, avec comparatif jour, semaine, mois.
4. Si la serveuse marque « servie » sans confirmation du client, la commande se clôt automatiquement après un délai réglable.

CONTRÔLES (en local uniquement, aucune CI) : Tests : compte à rebours cohérent après coupure réseau ; échéance dépassée = alerte ; statistiques de temps de service exactes.

LIVRABLES : Code complet avec chemins exacts, migrations SQL ordonnées (non appliquées), textes FR/EN, tests locaux demandés, et le fichier de passation.

CONTRAINTES DU PROJET :

- Android uniquement. Hors connexion d'abord : tout fonctionne sur la base locale, la synchronisation se fait au retour du réseau.
- Uniquement des solutions gratuites (Supabase offre gratuite partagée, Firebase Cloud Messaging, bibliothèques libres, GitHub). Aucune dépense sans mon accord.
- Sécurité : aucun code PIN ni OTP n'est lu, stocké, transmis ni affiché ; un SMS qui en contient est ignoré. Aucun message WhatsApp ne contient de PIN ou d'OTP.
- L'IA propose, l'humain décide : jamais de validation de paiement ni de modification de solde par l'IA seule ; chaque suggestion explique pourquoi ; aucun numéro ni SMS brut envoyé à un service externe.
- Aucun texte en dur dans le code : tout est dans des fichiers de langue français et English. Montants en francs CFA, formatés (45 000 F), calculs exacts au franc près.
- Les accès clients sont créés uniquement par l'administration ; pas d'inscription libre dans l'application. Un seul backend Supabase partagé par les 7 applications, avec règles RLS strictes.
- L'application suit la charte graphique de son logo (couleurs, police, thème) ; états toujours doublés d'une icône et d'un libellé ; contraste texte d'au moins 4,5 : 1.
- Les pages vues par les clients finaux (menu par QR, suivi de service, paiement) sont des pages web légères : aucune installation, moins de 150 Ko au premier chargement, utilisables sur réseau lent, aux couleurs de l'application, en français et en English.
- Ne jamais faire confiance au téléphone du client : prix, totaux, disponibilité et statuts sont recalculés côté serveur ; limitation du nombre de commandes par appareil et par table ; code de table à saisir ; un client ne voit que sa propre commande grâce à un jeton aléatoire.
- Aucune donnée personnelle obligatoire pour le client final (le numéro de téléphone est facultatif et accompagné d'un consentement clair) ; aucune donnée sensible dans les liens.
- Pas d'envoi automatique payant : WhatsApp reste en message préparé envoyé en un clic, avec une couche d'envoi remplaçable (un prestataire payant pourra être branché plus tard sans réécrire l'application). Les notifications gratuites passent par la page web du client (mise à jour en direct et notification web si le client

l'accepte).

- Ne jamais inventer un code USSD de paiement : demande-moi les codes exacts d'Orange Money et de MTN MoMo pour mon restaurant et mes commerces avant de coder les liens de paiement.
- Tout calcul de stock est atomique et idempotent : une même commande ne peut ni retirer deux fois le stock, ni le restituer deux fois ; jamais de stock négatif silencieux.

RÈGLES DE TRAVAIL OBLIGATOIRES :

1. Travaille uniquement sur la branche **main**. Ne crée aucune branche parallèle.
2. N'ouvre aucune pull request (PR).
3. Ne lance, ne déclenche et n'attends aucune vérification CI (aucun workflow GitHub Actions). Les tests se font en local uniquement, et seulement ceux que ce prompt demande.
4. Ne fais aucun déploiement : ni publication d'APK ou de release, ni migration appliquée à Supabase, ni fonction serveur déployée, ni mise en ligne d'une page. Prépare seulement les fichiers.
5. Tout ce qui précède est interdit **sans mon accord explicite, écrit dans cette conversation, pour chaque action concernée**. Si une étape semble l'exiger (y compris un push qui déclencherait un workflow automatique), arrête-toi, explique pourquoi et demande-moi.
6. Si tu n'as pas accès à mon dépôt, livre les fichiers complets avec leur chemin exact, prêts à copier ; ne suppose pas qu'ils existent déjà.
7. N'invente pas : si une information manque, pose-moi la question (une seule à la fois) avant de continuer.

MODÈLE DU FICHIER DE PASSATION À PRODUIRE EN FIN DE RÉPONSE :

FICHIER DE PASSATION — [application] — [numéro du prompt] — [date]

1. Fait : ce qui est terminé, en une ligne chacun.
2. Fichiers créés ou modifiés : chemins exacts.
3. Tables, fonctions et écrans créés : noms exacts.
4. Décisions techniques retenues et leur raison.
5. Points ouverts et questions pour moi.
6. Prochain prompt recommandé (numéro et titre).
7. Actions qui demandent mon accord explicite (push, migration, déploiement…).

Termine par : « Étape terminée. J'attends ton accord avant de passer à la suivante. »

PROMPT 10.5 — NAXA Services : prévenir le client quand son service est prêt

RÔLE : Tu es un ingénieur logiciel senior (Android, Supabase et PostgreSQL, sécurité, interfaces mobiles pour téléphones d'entrée de gamme). Je suis porteur de projet et non développeur : explique brièvement, puis livre du code complet et prêt à l'emploi.

PROJET : Fonctionnalités novatrices à ajouter à la famille d'applications, à exécuter UNIQUEMENT après tous les autres prompts, après la refonte du design (série 8) et idéalement après le module Caméras (série 9). Contenu : (1) Zipple : commande par QR de table avec paiement mobile money ou espèces, validation par la serveuse, compte à rebours de service, confirmation « servi », et stock mis à jour automatiquement ; (2) NAXA Services : prévenir le client par WhatsApp quand son service est prêt ; (3) QR « Payer ici » dans les applications qui encaissent ; (4) import d'un cahier papier par photo ; (5) attestation d'activité vérifiable ; (6) CotiZ : étude puis prototype d'un assistant vocal en langue locale. Les applications existent déjà : réutilise leurs modules (lecture des SMS, stock, recettes, notifications, thème) au lieu de les réécrire.

ÉTAT DE DÉPART : voici le fichier de passation de l'étape précédente : [COLLE ICI LE FICHIER DE PASSATION]. S'il est vide ou absent, dis-le-moi avant de commencer.

MISSION : Informer le client par WhatsApp que son service ou son linge est prêt.

À FAIRE :

1. Quand le statut passe à « prêt » (pressing) ou quand un rendez-vous se termine, l'application affiche « Prévenir le client » et ouvre WhatsApp avec un message prérempli (nom, numéro de ticket, montant restant, horaires de retrait), à envoyer en un clic. Liste « clients à prévenir » avec file d'envoi pour traiter plusieurs clients à la suite.
2. Couche d'envoi remplaçable (interface MessageSender) : implémentation actuelle = lien WhatsApp préparé ; une implémentation payante (WhatsApp Business via un prestataire) pourra être branchée plus tard sans réécrire l'application. N'implémente PAS l'envoi payant sans mon accord.
3. Page de suivi pour le client (lien ou QR sur le ticket, sans installation) : statut en direct (reçu, en cours, prêt, retiré), montant restant, horaires ; notification web si le client l'accepte.
4. Modèles de messages modifiables en français et en English ; jamais de PIN ou d'OTP ; consentement du client pour être contacté ; rappel automatique proposé si le linge n'est pas retiré après un délai réglable.

CONTRÔLES (en local uniquement, aucune CI) : Tests : passage à « prêt » prépare le message correct ; lien de suivi affiche le bon statut.

LIVRABLES : Code complet avec chemins exacts, migrations SQL ordonnées (non appliquées), textes FR/EN, tests locaux demandés, et le fichier de passation.

CONTRAINTES DU PROJET :

- Android uniquement. Hors connexion d'abord : tout fonctionne sur la base locale, la synchronisation se fait au retour du réseau.
- Uniquement des solutions gratuites (Supabase offre gratuite partagée, Firebase Cloud Messaging, bibliothèques libres, GitHub). Aucune dépense sans mon accord.
- Sécurité : aucun code PIN ni OTP n'est lu, stocké, transmis ni affiché ; un SMS qui en contient est ignoré. Aucun message WhatsApp ne contient de PIN ou d'OTP.
- L'IA propose, l'humain décide : jamais de validation de paiement ni de modification de solde par l'IA seule ; chaque suggestion explique pourquoi ; aucun numéro ni SMS brut envoyé à un service externe.
- Aucun texte en dur dans le code : tout est dans des fichiers de langue français et English. Montants en francs CFA, formatés (45 000 F), calculs exacts au franc près.
- Les accès clients sont créés uniquement par l'administration ; pas d'inscription libre dans l'application. Un seul backend Supabase partagé par les 7 applications, avec règles RLS strictes.
- L'application suit la charte graphique de son logo (couleurs, police, thème) ; états toujours doublés d'une icône et d'un libellé ; contraste texte d'au moins 4,5 : 1.
- Les pages vues par les clients finaux (menu par QR, suivi de service, paiement) sont des pages web légères : aucune installation, moins de 150 Ko au premier chargement, utilisables sur réseau lent, aux couleurs de l'application, en français et en English.
- Ne jamais faire confiance au téléphone du client : prix, totaux, disponibilité et statuts sont recalculés côté serveur ; limitation du nombre de commandes par appareil et par table ; code de table à saisir ; un client ne voit que sa propre commande grâce à un jeton aléatoire.
- Aucune donnée personnelle obligatoire pour le client final (le numéro de téléphone est facultatif et accompagné d'un consentement clair) ; aucune donnée sensible dans les liens.
- Pas d'envoi automatique payant : WhatsApp reste en message préparé envoyé en un clic, avec une couche d'envoi remplaçable (un prestataire payant pourra être branché plus tard sans réécrire l'application). Les notifications gratuites passent par la page web du client (mise à jour en direct et notification web si le client l'accepte).
- Ne jamais inventer un code USSD de paiement : demande-moi les codes exacts d'Orange Money et de MTN MoMo pour mon restaurant et mes commerces avant de coder les liens de paiement.
- Tout calcul de stock est atomique et idempotent : une même commande ne peut ni retirer deux fois le stock, ni le restituer deux fois ; jamais de stock négatif silencieux.

RÈGLES DE TRAVAIL OBLIGATOIRES :

1. Travaille uniquement sur la branche **main**. Ne crée aucune branche parallèle.
2. N'ouvre aucune pull request (PR).
3. Ne lance, ne déclenche et n'attends aucune vérification CI (aucun workflow GitHub Actions). Les tests se font en local uniquement, et seulement ceux que ce prompt demande.
4. Ne fais aucun déploiement : ni publication d'APK ou de release, ni migration appliquée à Supabase, ni fonction serveur déployée, ni mise en ligne d'une page. Prépare seulement les fichiers.
5. Tout ce qui précède est interdit **sans mon accord explicite, écrit dans cette conversation, pour chaque action concernée**. Si une étape semble l'exiger (y compris un push qui déclencherait un workflow automatique), arrête-toi, explique pourquoi et demande-moi.
6. Si tu n'as pas accès à mon dépôt, livre les fichiers complets avec leur chemin exact, prêts à copier ; ne suppose pas qu'ils existent déjà.
7. N'invente pas : si une information manque, pose-moi la question (une seule à la fois) avant de continuer.

MODÈLE DU FICHIER DE PASSATION À PRODUIRE EN FIN DE RÉPONSE :

FICHIER DE PASSATION — [application] — [numéro du prompt] — [date]

1. Fait : ce qui est terminé, en une ligne chacun.
2. Fichiers créés ou modifiés : chemins exacts.
3. Tables, fonctions et écrans créés : noms exacts.
4. Décisions techniques retenues et leur raison.
5. Points ouverts et questions pour moi.
6. Prochain prompt recommandé (numéro et titre).
7. Actions qui demandent mon accord explicite (push, migration, déploiement…).

Termine par : « Étape terminée. J'attends ton accord avant de passer à la suivante. »

PROMPT 10.6 — QR « Payer ici » (applications qui encaissent)

RÔLE : Tu es un ingénieur logiciel senior (Android, Supabase et PostgreSQL, sécurité, interfaces mobiles pour téléphones d'entrée de gamme). Je suis porteur de projet et non développeur : explique brièvement, puis livre du code complet et prêt à l'emploi.

PROJET : Fonctionnalités novatrices à ajouter à la famille d'applications, à exécuter UNIQUEMENT après tous les autres prompts, après la refonte du design (série 8) et idéalement après le module Caméras (série 9). Contenu : (1) Zipple : commande par QR de table avec paiement mobile money ou espèces, validation par la serveuse, compte à rebours de service, confirmation « servi », et stock mis à jour automatiquement ; (2) NAXA Services : prévenir le client par WhatsApp quand son service est prêt ; (3) QR « Payer ici » dans les applications qui encaissent ; (4) import d'un cahier papier par photo ; (5) attestation d'activité vérifiable ; (6) CotiZ : étude puis prototype d'un assistant vocal en langue locale. Les applications existent déjà : réutilise leurs modules (lecture des SMS, stock, recettes, notifications, thème) au lieu de les réécrire.

ÉTAT DE DÉPART : voici le fichier de passation de l'étape précédente : [COLLE ICI LE FICHIER DE PASSATION]. S'il est vide ou absent, dis-le-moi avant de commencer.

MISSION : Permettre au client de payer en mobile money en scannant un QR au comptoir, avec confirmation automatique.

À FAIRE :

1. Applications concernées : Bootik, Zipple, NAXA Services, Loyable, NAXA Hotel, CotiZ. Un module commun génère un QR par point d'encaissement (ou par facture, par loyer, par cotisation, par acompte) ; le montant vient de l'application (total de vente, loyer dû, acompte, cotisation).
2. Le QR ouvre une page web légère : montant, référence unique, boutons de paiement Orange Money et MTN MoMo qui préparent le paiement vers mon code marchand sur le téléphone du client. Demande-moi les codes USSD exacts ; ne les invente pas. Plan B : afficher le numéro marchand, le montant et la référence à copier.
3. Confirmation : la lecture des SMS existante rapproche le paiement (montant exact, référence, horaire) et marque la vente, le loyer ou la cotisation « payé(e) » ; la page du client passe à « Paiement reçu » avec reçu partageable.
4. Sécurité : référence à usage unique, expiration réglable, montant imposé par le serveur, jamais de PIN ni d'OTP, un même paiement ne solde qu'un seul élément.

CONTRÔLES (en local uniquement, aucune CI) : Tests : paiement reconnu = élément soldé sans doublon ; QR expiré refusé ; montant falsifié refusé.

LIVRABLES : Code complet avec chemins exacts, migrations SQL ordonnées (non appliquées), textes FR/EN, tests locaux demandés, et le fichier de passation.

CONTRAINTES DU PROJET :

- Android uniquement. Hors connexion d'abord : tout fonctionne sur la base locale, la synchronisation se fait au retour du réseau.
- Uniquement des solutions gratuites (Supabase offre gratuite partagée, Firebase Cloud Messaging, bibliothèques libres, GitHub). Aucune dépense sans mon accord.
- Sécurité : aucun code PIN ni OTP n'est lu, stocké, transmis ni affiché ; un SMS qui en contient est ignoré. Aucun message WhatsApp ne contient de PIN ou d'OTP.
- L'IA propose, l'humain décide : jamais de validation de paiement ni de modification de solde par l'IA seule ; chaque suggestion explique pourquoi ; aucun numéro ni SMS brut envoyé à un service externe.
- Aucun texte en dur dans le code : tout est dans des fichiers de langue français et English. Montants en francs CFA, formatés (45 000 F), calculs exacts au franc près.
- Les accès clients sont créés uniquement par l'administration ; pas d'inscription libre dans l'application. Un seul backend Supabase partagé par les 7 applications, avec règles RLS strictes.
- L'application suit la charte graphique de son logo (couleurs, police, thème) ; états toujours doublés d'une icône et d'un libellé ; contraste texte d'au moins 4,5 : 1.
- Les pages vues par les clients finaux (menu par QR, suivi de service, paiement) sont des pages web légères : aucune installation, moins de 150 Ko au premier chargement, utilisables sur réseau lent, aux couleurs de l'application, en français et en English.
- Ne jamais faire confiance au téléphone du client : prix, totaux, disponibilité et statuts sont recalculés côté serveur ; limitation du nombre de commandes par appareil et par table ; code de table à saisir ; un client ne voit que sa propre commande grâce à un jeton aléatoire.
- Aucune donnée personnelle obligatoire pour le client final (le numéro de téléphone est facultatif et accompagné d'un consentement clair) ; aucune donnée sensible dans les liens.
- Pas d'envoi automatique payant : WhatsApp reste en message préparé envoyé en un clic, avec une couche d'envoi remplaçable (un prestataire payant pourra être branché plus tard sans réécrire l'application). Les

notifications gratuites passent par la page web du client (mise à jour en direct et notification web si le client l'accepte).

- Ne jamais inventer un code USSD de paiement : demande-moi les codes exacts d'Orange Money et de MTN MoMo pour mon restaurant et mes commerces avant de coder les liens de paiement.
- Tout calcul de stock est atomique et idempotent : une même commande ne peut ni retirer deux fois le stock, ni le restituer deux fois ; jamais de stock négatif silencieux.

RÈGLES DE TRAVAIL OBLIGATOIRES :

1. Travaille uniquement sur la branche **main**. Ne crée aucune branche parallèle.
2. N'ouvre aucune pull request (PR).
3. Ne lance, ne déclenche et n'attends aucune vérification CI (aucun workflow GitHub Actions). Les tests se font en local uniquement, et seulement ceux que ce prompt demande.
4. Ne fais aucun déploiement : ni publication d'APK ou de release, ni migration appliquée à Supabase, ni fonction serveur déployée, ni mise en ligne d'une page. Prépare seulement les fichiers.
5. Tout ce qui précède est interdit **sans mon accord explicite, écrit dans cette conversation, pour chaque action concernée**. Si une étape semble l'exiger (y compris un push qui déclencherait un workflow automatique), arrête-toi, explique pourquoi et demande-moi.
6. Si tu n'as pas accès à mon dépôt, livre les fichiers complets avec leur chemin exact, prêts à copier ; ne suppose pas qu'ils existent déjà.
7. N'invente pas : si une information manque, pose-moi la question (une seule à la fois) avant de continuer.

MODÈLE DU FICHIER DE PASSATION À PRODUIRE EN FIN DE RÉPONSE :

FICHIER DE PASSATION — [application] — [numéro du prompt] — [date]

1. Fait : ce qui est terminé, en une ligne chacun.
2. Fichiers créés ou modifiés : chemins exacts.
3. Tables, fonctions et écrans créés : noms exacts.
4. Décisions techniques retenues et leur raison.
5. Points ouverts et questions pour moi.
6. Prochain prompt recommandé (numéro et titre).
7. Actions qui demandent mon accord explicite (push, migration, déploiement…).

Termine par : « Étape terminée. J'attends ton accord avant de passer à la suivante. »

PROMPT 10.7 — Import d'un cahier papier par photo

RÔLE : Tu es un ingénieur logiciel senior (Android, Supabase et PostgreSQL, sécurité, interfaces mobiles pour téléphones d'entrée de gamme). Je suis porteur de projet et non développeur : explique brièvement, puis livre du code complet et prêt à l'emploi.

PROJET : Fonctionnalités novatrices à ajouter à la famille d'applications, à exécuter UNIQUEMENT après tous les autres prompts, après la refonte du design (série 8) et idéalement après le module Caméras (série 9). Contenu : (1) Zipple : commande par QR de table avec paiement mobile money ou espèces, validation par la serveuse, compte à rebours de service, confirmation « servi », et stock mis à jour automatiquement ; (2) NAXA Services : prévenir le client par WhatsApp quand son service est prêt ; (3) QR « Payer ici » dans les applications qui encaissent ; (4) import d'un cahier papier par photo ; (5) attestation d'activité vérifiable ; (6) CotiZ : étude puis prototype d'un assistant vocal en langue locale. Les applications existent déjà : réutilise leurs modules (lecture des SMS, stock, recettes, notifications, thème) au lieu de les réécrire.

ÉTAT DE DÉPART : voici le fichier de passation de l'étape précédente : [COLLE ICI LE FICHIER DE PASSATION]. S'il est vide ou absent, dis-le-moi avant de commencer.

MISSION : Aider un nouveau client à reprendre son cahier d'ardoises, de stock ou de loyers.

À FAIRE :

1. Le gérant photographie une page de son cahier ; reconnaissance de texte SUR LE TÉLÉPHONE (bibliothèque libre, sans envoi de l'image) ; l'application propose des lignes (nom, montant, date, quantité) selon le type choisi : ardoises et dettes, liste de produits et stock, loyers, cotisations, clients.
2. Écran de validation obligatoire : chaque ligne est affichée à côté de l'extrait de la photo, corrigeable, avec un indice de confiance ; rien n'est enregistré sans validation ; import par lots avec annulation.
3. Limites à afficher honnêtement : l'écriture manuscrite est mal lue ; conseille d'écrire les chiffres lisiblement ; propose un modèle de page imprimable pour de meilleurs résultats. L'image est supprimée après l'import.
4. Disponible dans Bootik, Zipple, NAXA Services, Loyable et CotiZ selon le type de données.

CONTRÔLES (en local uniquement, aucune CI) : Tests sur 20 photos d'exemple : taux de lignes correctes mesuré et rapporté ; aucune écriture sans validation.

LIVRABLES : Code complet avec chemins exacts, migrations SQL ordonnées (non appliquées), textes FR/EN, tests locaux demandés, et le fichier de passation.

CONTRAINTES DU PROJET :

- Android uniquement. Hors connexion d'abord : tout fonctionne sur la base locale, la synchronisation se fait au retour du réseau.
- Uniquement des solutions gratuites (Supabase offre gratuite partagée, Firebase Cloud Messaging, bibliothèques libres, GitHub). Aucune dépense sans mon accord.
- Sécurité : aucun code PIN ni OTP n'est lu, stocké, transmis ni affiché ; un SMS qui en contient est ignoré. Aucun message WhatsApp ne contient de PIN ou d'OTP.
- L'IA propose, l'humain décide : jamais de validation de paiement ni de modification de solde par l'IA seule ; chaque suggestion explique pourquoi ; aucun numéro ni SMS brut envoyé à un service externe.
- Aucun texte en dur dans le code : tout est dans des fichiers de langue français et English. Montants en francs CFA, formatés (45 000 F), calculs exacts au franc près.
- Les accès clients sont créés uniquement par l'administration ; pas d'inscription libre dans l'application. Un seul backend Supabase partagé par les 7 applications, avec règles RLS strictes.
- L'application suit la charte graphique de son logo (couleurs, police, thème) ; états toujours doublés d'une icône et d'un libellé ; contraste texte d'au moins 4,5 : 1.
- Les pages vues par les clients finaux (menu par QR, suivi de service, paiement) sont des pages web légères : aucune installation, moins de 150 Ko au premier chargement, utilisables sur réseau lent, aux couleurs de l'application, en français et en English.
- Ne jamais faire confiance au téléphone du client : prix, totaux, disponibilité et statuts sont recalculés côté serveur ; limitation du nombre de commandes par appareil et par table ; code de table à saisir ; un client ne voit que sa propre commande grâce à un jeton aléatoire.
- Aucune donnée personnelle obligatoire pour le client final (le numéro de téléphone est facultatif et accompagné d'un consentement clair) ; aucune donnée sensible dans les liens.
- Pas d'envoi automatique payant : WhatsApp reste en message préparé envoyé en un clic, avec une couche d'envoi remplaçable (un prestataire payant pourra être branché plus tard sans réécrire l'application). Les notifications gratuites passent par la page web du client (mise à jour en direct et notification web si le client l'accepte).
- Ne jamais inventer un code USSD de paiement : demande-moi les codes exacts d'Orange Money et de MTN MoMo pour mon restaurant et mes commerces avant de coder les liens de paiement.
- Tout calcul de stock est atomique et idempotent : une même commande ne peut ni retirer deux fois le stock, ni le restituer deux fois ; jamais de stock négatif silencieux.

RÈGLES DE TRAVAIL OBLIGATOIRES :

1. Travaille uniquement sur la branche **main**. Ne crée aucune branche parallèle.
2. N'ouvre aucune pull request (PR).
3. Ne lance, ne déclenche et n'attends aucune vérification CI (aucun workflow GitHub Actions). Les tests se font en local uniquement, et seulement ceux que ce prompt demande.
4. Ne fais aucun déploiement : ni publication d'APK ou de release, ni migration appliquée à Supabase, ni fonction serveur déployée, ni mise en ligne d'une page. Prépare seulement les fichiers.
5. Tout ce qui précède est interdit **sans mon accord explicite, écrit dans cette conversation, pour chaque action concernée**. Si une étape semble l'exiger (y compris un push qui déclencherait un workflow automatique), arrête-toi, explique pourquoi et demande-moi.
6. Si tu n'as pas accès à mon dépôt, livre les fichiers complets avec leur chemin exact, prêts à copier ; ne suppose pas qu'ils existent déjà.
7. N'invente pas : si une information manque, pose-moi la question (une seule à la fois) avant de continuer.

MODÈLE DU FICHIER DE PASSATION À PRODUIRE EN FIN DE RÉPONSE :

FICHIER DE PASSATION — [application] — [numéro du prompt] — [date]

1. Fait : ce qui est terminé, en une ligne chacun.
2. Fichiers créés ou modifiés : chemins exacts.
3. Tables, fonctions et écrans créés : noms exacts.
4. Décisions techniques retenues et leur raison.
5. Points ouverts et questions pour moi.
6. Prochain prompt recommandé (numéro et titre).
7. Actions qui demandent mon accord explicite (push, migration, déploiement…).

Termine par : « Étape terminée. J'attends ton accord avant de passer à la suivante. »

PROMPT 10.8 — Attestation d'activité vérifiable

RÔLE : Tu es un ingénieur logiciel senior (Android, Supabase et PostgreSQL, sécurité, interfaces mobiles pour téléphones d'entrée de gamme). Je suis porteur de projet et non développeur : explique brièvement, puis livre du code complet et prêt à l'emploi.

PROJET : Fonctionnalités novatrices à ajouter à la famille d'applications, à exécuter UNIQUEMENT après tous les autres prompts, après la refonte du design (série 8) et idéalement après le module Caméras (série 9). Contenu : (1) Zipple : commande par QR de table avec paiement mobile money ou espèces, validation par la serveuse, compte à rebours de service, confirmation « servi », et stock mis à jour automatiquement ; (2) NAXA Services : prévenir le client par WhatsApp quand son service est prêt ; (3) QR « Payer ici » dans les applications qui encaissent ; (4) import d'un cahier papier par photo ; (5) attestation d'activité vérifiable ; (6) CotiZ : étude puis prototype d'un assistant vocal en langue locale. Les applications existent déjà : réutilise leurs modules (lecture des SMS, stock, recettes, notifications, thème) au lieu de les réécrire.

ÉTAT DE DÉPART : voici le fichier de passation de l'étape précédente : [COLLE ICI LE FICHIER DE PASSATION]. S'il est vide ou absent, dis-le-moi avant de commencer.

MISSION : Produire un document certifié de l'activité d'un commerce, que le client peut montrer à un partenaire.

À FAIRE :

1. Document PDF sur une période choisie : volume d'activité, nombre d'opérations, régularité par mois, sans le détail des clients ni des transactions individuelles. Un code QR renvoie vers une page de vérification qui confirme que le document est authentique (empreinte du contenu enregistrée côté serveur).
2. Consentement explicite du client à chaque génération ; durée de validité affichée ; aucune note de crédit ni promesse de financement : le document atteste seulement l'activité enregistrée par l'application.
3. Disponible dans Kiosko, Bootik, Zipple, NAXA Services, Loyable et NAXA Hotel ; mise en garde : l'usage par une banque ou une institution de microfinance dépend d'accords et d'un cadre légal à faire valider avant toute promesse commerciale.

CONTRÔLES (en local uniquement, aucune CI) : Tests : document modifié = vérification échoue ; document authentique = vérification réussit.

LIVRABLES : Code complet avec chemins exacts, migrations SQL ordonnées (non appliquées), textes FR/EN, tests locaux demandés, et le fichier de passation.

CONTRAINTES DU PROJET :

- Android uniquement. Hors connexion d'abord : tout fonctionne sur la base locale, la synchronisation se fait au retour du réseau.
- Uniquement des solutions gratuites (Supabase offre gratuite partagée, Firebase Cloud Messaging, bibliothèques libres, GitHub). Aucune dépense sans mon accord.
- Sécurité : aucun code PIN ni OTP n'est lu, stocké, transmis ni affiché ; un SMS qui en contient est ignoré. Aucun message WhatsApp ne contient de PIN ou d'OTP.
- L'IA propose, l'humain décide : jamais de validation de paiement ni de modification de solde par l'IA seule ; chaque suggestion explique pourquoi ; aucun numéro ni SMS brut envoyé à un service externe.
- Aucun texte en dur dans le code : tout est dans des fichiers de langue français et English. Montants en francs CFA, formatés (45 000 F), calculs exacts au franc près.
- Les accès clients sont créés uniquement par l'administration ; pas d'inscription libre dans l'application. Un seul backend Supabase partagé par les 7 applications, avec règles RLS strictes.
- L'application suit la charte graphique de son logo (couleurs, police, thème) ; états toujours doublés d'une icône et d'un libellé ; contraste texte d'au moins 4,5 : 1.
- Les pages vues par les clients finaux (menu par QR, suivi de service, paiement) sont des pages web légères : aucune installation, moins de 150 Ko au premier chargement, utilisables sur réseau lent, aux couleurs de l'application, en français et en English.
- Ne jamais faire confiance au téléphone du client : prix, totaux, disponibilité et statuts sont recalculés côté serveur ; limitation du nombre de commandes par appareil et par table ; code de table à saisir ; un client ne voit que sa propre commande grâce à un jeton aléatoire.
- Aucune donnée personnelle obligatoire pour le client final (le numéro de téléphone est facultatif et accompagné d'un consentement clair) ; aucune donnée sensible dans les liens.
- Pas d'envoi automatique payant : WhatsApp reste en message préparé envoyé en un clic, avec une couche d'envoi remplaçable (un prestataire payant pourra être branché plus tard sans réécrire l'application). Les notifications gratuites passent par la page web du client (mise à jour en direct et notification web si le client l'accepte).
- Ne jamais inventer un code USSD de paiement : demande-moi les codes exacts d'Orange Money et de MTN MoMo pour mon restaurant et mes commerces avant de coder les liens de paiement.
- Tout calcul de stock est atomique et idempotent : une même commande ne peut ni retirer deux fois le stock, ni le restituer deux fois ; jamais de stock négatif silencieux.

RÈGLES DE TRAVAIL OBLIGATOIRES :

1. Travaille uniquement sur la branche **main**. Ne crée aucune branche parallèle.
2. N'ouvre aucune pull request (PR).
3. Ne lance, ne déclenche et n'attends aucune vérification CI (aucun workflow GitHub Actions). Les tests se font en local uniquement, et seulement ceux que ce prompt demande.
4. Ne fais aucun déploiement : ni publication d'APK ou de release, ni migration appliquée à Supabase, ni fonction serveur déployée, ni mise en ligne d'une page. Prépare seulement les fichiers.
5. Tout ce qui précède est interdit **sans mon accord explicite, écrit dans cette conversation, pour chaque action concernée**. Si une étape semble l'exiger (y compris un push qui déclencherait un workflow automatique), arrête-toi, explique pourquoi et demande-moi.
6. Si tu n'as pas accès à mon dépôt, livre les fichiers complets avec leur chemin exact, prêts à copier ; ne suppose pas qu'ils existent déjà.
7. N'invente pas : si une information manque, pose-moi la question (une seule à la fois) avant de continuer.

MODÈLE DU FICHIER DE PASSATION À PRODUIRE EN FIN DE RÉPONSE :

FICHIER DE PASSATION — [application] — [numéro du prompt] — [date]

1. Fait : ce qui est terminé, en une ligne chacun.
2. Fichiers créés ou modifiés : chemins exacts.
3. Tables, fonctions et écrans créés : noms exacts.
4. Décisions techniques retenues et leur raison.
5. Points ouverts et questions pour moi.
6. Prochain prompt recommandé (numéro et titre).
7. Actions qui demandent mon accord explicite (push, migration, déploiement…).

Termine par : « Étape terminée. J'attends ton accord avant de passer à la suivante. »

PROMPT 10.9 — CotiZ : assistant vocal en langue locale (étude puis prototype)

RÔLE : Tu es un ingénieur logiciel senior (Android, Supabase et PostgreSQL, sécurité, interfaces mobiles pour téléphones d'entrée de gamme). Je suis porteur de projet et non développeur : explique brièvement, puis livre du code complet et prêt à l'emploi.

PROJET : Fonctionnalités novatrices à ajouter à la famille d'applications, à exécuter UNIQUEMENT après tous les autres prompts, après la refonte du design (série 8) et idéalement après le module Caméras (série 9). Contenu : (1) Zipple : commande par QR de table avec paiement mobile money ou espèces, validation par la serveuse, compte à rebours de service, confirmation « servi », et stock mis à jour automatiquement ; (2) NAXA Services : prévenir le client par WhatsApp quand son service est prêt ; (3) QR « Payer ici » dans les applications qui encaissent ; (4) import d'un cahier papier par photo ; (5) attestation d'activité vérifiable ; (6) CotiZ : étude puis prototype d'un assistant vocal en langue locale. Les applications existent déjà : réutilise leurs modules (lecture des SMS, stock, recettes, notifications, thème) au lieu de les réécrire.

ÉTAT DE DÉPART : voici le fichier de passation de l'étape précédente : [COLLE ICI LE FICHIER DE PASSATION]. S'il est vide ou absent, dis-le-moi avant de commencer.

MISSION : Évaluer honnêtement ce qui est possible, puis proposer un prototype limité.

À FAIRE :

1. ÉTAPE 1, rapport seulement : quelles langues camerounaises (par exemple ewondo, duala, bassa, fulfulde, ghomala, et le camfranglais) disposent de reconnaissance vocale gratuite exploitable ? Quelle qualité réelle ? Quelles données vocales libres existent ? Ne promets rien sans source ; indique les incertitudes. Arrête-toi et attends mon accord avant l'étape 2.
2. ÉTAPE 2 (après mon accord) : commandes vocales en français et en English avec la reconnaissance vocale d'Android pour des phrases simples (« combien me doit Marie ? », « qui est en retard ? ») avec affichage du résultat pour confirmation.
3. ÉTAPE 3 (après mon accord) : messages audio enregistrés par des locuteurs natifs pour les notifications clés (cotisation due, tour, réunion) dans les langues choisies ; reconnaissance d'un petit vocabulaire fermé par mots-clés, entraînée sur des échantillons collectés avec consentement. Ne promets pas la compréhension libre de la parole.

CONTRÔLES (en local uniquement, aucune CI) : Rapport d'étude livré ; aucune fonction vocale en langue locale codée sans mon accord écrit.

LIVRABLES : Code complet avec chemins exacts, migrations SQL ordonnées (non appliquées), textes FR/EN, tests locaux demandés, et le fichier de passation.

CONTRAINTES DU PROJET :

- Android uniquement. Hors connexion d'abord : tout fonctionne sur la base locale, la synchronisation se fait au retour du réseau.
- Uniquement des solutions gratuites (Supabase offre gratuite partagée, Firebase Cloud Messaging, bibliothèques libres, GitHub). Aucune dépense sans mon accord.
- Sécurité : aucun code PIN ni OTP n'est lu, stocké, transmis ni affiché ; un SMS qui en contient est ignoré. Aucun message WhatsApp ne contient de PIN ou d'OTP.
- L'IA propose, l'humain décide : jamais de validation de paiement ni de modification de solde par l'IA seule ; chaque suggestion explique pourquoi ; aucun numéro ni SMS brut envoyé à un service externe.
- Aucun texte en dur dans le code : tout est dans des fichiers de langue français et English. Montants en francs CFA, formatés (45 000 F), calculs exacts au franc près.
- Les accès clients sont créés uniquement par l'administration ; pas d'inscription libre dans l'application. Un seul backend Supabase partagé par les 7 applications, avec règles RLS strictes.
- L'application suit la charte graphique de son logo (couleurs, police, thème) ; états toujours doublés d'une icône et d'un libellé ; contraste texte d'au moins 4,5 : 1.
- Les pages vues par les clients finaux (menu par QR, suivi de service, paiement) sont des pages web légères : aucune installation, moins de 150 Ko au premier chargement, utilisables sur réseau lent, aux couleurs de l'application, en français et en English.
- Ne jamais faire confiance au téléphone du client : prix, totaux, disponibilité et statuts sont recalculés côté serveur ; limitation du nombre de commandes par appareil et par table ; code de table à saisir ; un client ne voit que sa propre commande grâce à un jeton aléatoire.
- Aucune donnée personnelle obligatoire pour le client final (le numéro de téléphone est facultatif et accompagné d'un consentement clair) ; aucune donnée sensible dans les liens.
- Pas d'envoi automatique payant : WhatsApp reste en message préparé envoyé en un clic, avec une couche d'envoi remplaçable (un prestataire payant pourra être branché plus tard sans réécrire l'application). Les notifications gratuites passent par la page web du client (mise à jour en direct et notification web si le client l'accepte).
- Ne jamais inventer un code USSD de paiement : demande-moi les codes exacts d'Orange Money et de MTN MoMo pour mon restaurant et mes commerces avant de coder les liens de paiement.
- Tout calcul de stock est atomique et idempotent : une même commande ne peut ni retirer deux fois le stock, ni le restituer deux fois ; jamais de stock négatif silencieux.

RÈGLES DE TRAVAIL OBLIGATOIRES :

1. Travaille uniquement sur la branche **main**. Ne crée aucune branche parallèle.
2. N'ouvre aucune pull request (PR).
3. Ne lance, ne déclenche et n'attends aucune vérification CI (aucun workflow GitHub Actions). Les tests se font en local uniquement, et seulement ceux que ce prompt demande.
4. Ne fais aucun déploiement : ni publication d'APK ou de release, ni migration appliquée à Supabase, ni fonction serveur déployée, ni mise en ligne d'une page. Prépare seulement les fichiers.
5. Tout ce qui précède est interdit **sans mon accord explicite, écrit dans cette conversation, pour chaque action concernée**. Si une étape semble l'exiger (y compris un push qui déclencherait un workflow automatique), arrête-toi, explique pourquoi et demande-moi.
6. Si tu n'as pas accès à mon dépôt, livre les fichiers complets avec leur chemin exact, prêts à copier ; ne suppose pas qu'ils existent déjà.
7. N'invente pas : si une information manque, pose-moi la question (une seule à la fois) avant de continuer.

MODÈLE DU FICHIER DE PASSATION À PRODUIRE EN FIN DE RÉPONSE :

FICHIER DE PASSATION — [application] — [numéro du prompt] — [date]

1. Fait : ce qui est terminé, en une ligne chacun.
2. Fichiers créés ou modifiés : chemins exacts.
3. Tables, fonctions et écrans créés : noms exacts.
4. Décisions techniques retenues et leur raison.
5. Points ouverts et questions pour moi.
6. Prochain prompt recommandé (numéro et titre).
7. Actions qui demandent mon accord explicite (push, migration, déploiement…).

Termine par : « Étape terminée. J'attends ton accord avant de passer à la suivante. »

PROMPT 10.10 — Recette des nouveautés

RÔLE : Tu es un ingénieur logiciel senior (Android, Supabase et PostgreSQL, sécurité, interfaces mobiles pour téléphones d'entrée de gamme). Je suis porteur de projet et non développeur : explique brièvement, puis livre du code complet et prêt à l'emploi.

PROJET : Fonctionnalités novatrices à ajouter à la famille d'applications, à exécuter UNIQUEMENT après tous les autres prompts, après la refonte du design (série 8) et idéalement après le module Caméras (série 9). Contenu : (1) Zipple : commande par QR de table avec paiement mobile money ou espèces, validation par la serveuse, compte à rebours de service, confirmation « servi », et stock mis à jour automatiquement ; (2) NAXA Services : prévenir le client par WhatsApp quand son service est prêt ; (3) QR « Payer ici » dans les applications qui encaissent ; (4) import d'un cahier papier par photo ; (5) attestation d'activité vérifiable ; (6) CotiZ : étude puis prototype d'un assistant vocal en langue locale. Les applications existent déjà : réutilise leurs modules (lecture des SMS, stock, recettes, notifications, thème) au lieu de les réécrire.

ÉTAT DE DÉPART : voici le fichier de passation de l'étape précédente : [COLLE ICI LE FICHIER DE PASSATION]. S'il est vide ou absent, dis-le-moi avant de commencer.

MISSION : Contrôler la sécurité, la fiabilité et la facilité des nouvelles fonctions.

À FAIRE :

1. Zipple : parcours complet client, serveuse et manager ; commande falsifiée, fausse table, spam de commandes, deux clients pour le dernier article, annulation et restitution du stock, paiement ambigu, téléchargement et partage du reçu (image, PDF, retéléchargement après plusieurs jours, authenticité par QR), coupure réseau côté serveuse et côté client ; page client sur téléphone d'entrée de gamme en réseau lent (poids, temps d'affichage).
2. NAXA Services, QR « Payer ici », import par photo, attestation : tests de bout en bout, cas d'erreur, confidentialité ; vérification qu'aucun paiement automatique payant ni aucun envoi payant n'a été activé.
3. Établis un rapport avec les écarts et leur priorité, et la liste des points qui demandent mon accord (codes USSD, prestataires, avis juridique). Ne publie rien.

CONTRÔLES (en local uniquement, aucune CI) : Rapport complet avec mesures ; aucun test fonctionnel ne régresse.

LIVRABLES : Code complet avec chemins exacts, migrations SQL ordonnées (non appliquées), textes FR/EN, tests locaux demandés, et le fichier de passation.

CONTRAINTES DU PROJET :

- Android uniquement. Hors connexion d'abord : tout fonctionne sur la base locale, la synchronisation se fait au retour du réseau.
- Uniquement des solutions gratuites (Supabase offre gratuite partagée, Firebase Cloud Messaging, bibliothèques libres, GitHub). Aucune dépense sans mon accord.
- Sécurité : aucun code PIN ni OTP n'est lu, stocké, transmis ni affiché ; un SMS qui en contient est ignoré. Aucun message WhatsApp ne contient de PIN ou d'OTP.
- L'IA propose, l'humain décide : jamais de validation de paiement ni de modification de solde par l'IA seule ; chaque suggestion explique pourquoi ; aucun numéro ni SMS brut envoyé à un service externe.
- Aucun texte en dur dans le code : tout est dans des fichiers de langue français et English. Montants en francs CFA, formatés (45 000 F), calculs exacts au franc près.
- Les accès clients sont créés uniquement par l'administration ; pas d'inscription libre dans l'application. Un seul backend Supabase partagé par les 7 applications, avec règles RLS strictes.
- L'application suit la charte graphique de son logo (couleurs, police, thème) ; états toujours doublés d'une icône et d'un libellé ; contraste texte d'au moins 4,5 : 1.
- Les pages vues par les clients finaux (menu par QR, suivi de service, paiement) sont des pages web légères : aucune installation, moins de 150 Ko au premier chargement, utilisables sur réseau lent, aux couleurs de l'application, en français et en English.
- Ne jamais faire confiance au téléphone du client : prix, totaux, disponibilité et statuts sont recalculés côté serveur ; limitation du nombre de commandes par appareil et par table ; code de table à saisir ; un client ne voit que sa propre commande grâce à un jeton aléatoire.
- Aucune donnée personnelle obligatoire pour le client final (le numéro de téléphone est facultatif et accompagné d'un consentement clair) ; aucune donnée sensible dans les liens.
- Pas d'envoi automatique payant : WhatsApp reste en message préparé envoyé en un clic, avec une couche d'envoi remplaçable (un prestataire payant pourra être branché plus tard sans réécrire l'application). Les notifications gratuites passent par la page web du client (mise à jour en direct et notification web si le client l'accepte).
- Ne jamais inventer un code USSD de paiement : demande-moi les codes exacts d'Orange Money et de MTN MoMo pour mon restaurant et mes commerces avant de coder les liens de paiement.
- Tout calcul de stock est atomique et idempotent : une même commande ne peut ni retirer deux fois le stock, ni le restituer deux fois ; jamais de stock négatif silencieux.

RÈGLES DE TRAVAIL OBLIGATOIRES :

1. Travaille uniquement sur la branche **main**. Ne crée aucune branche parallèle.
2. N'ouvre aucune pull request (PR).

3. Ne lance, ne déclenche et n'attends aucune vérification CI (aucun workflow GitHub Actions). Les tests se font en local uniquement, et seulement ceux que ce prompt demande.
4. Ne fais aucun déploiement : ni publication d'APK ou de release, ni migration appliquée à Supabase, ni fonction serveur déployée, ni mise en ligne d'une page. Prépare seulement les fichiers.
5. Tout ce qui précède est interdit **sans mon accord explicite, écrit dans cette conversation, pour chaque action concernée**. Si une étape semble l'exiger (y compris un push qui déclencherait un workflow automatique), arrête-toi, explique pourquoi et demande-moi.
6. Si tu n'as pas accès à mon dépôt, livre les fichiers complets avec leur chemin exact, prêts à copier ; ne suppose pas qu'ils existent déjà.
7. N'invente pas : si une information manque, pose-moi la question (une seule à la fois) avant de continuer.

MODÈLE DU FICHIER DE PASSATION À PRODUIRE EN FIN DE RÉPONSE :

FICHIER DE PASSATION — [application] — [numéro du prompt] — [date]

1. Fait : ce qui est terminé, en une ligne chacun.
2. Fichiers créés ou modifiés : chemins exacts.
3. Tables, fonctions et écrans créés : noms exacts.
4. Décisions techniques retenues et leur raison.
5. Points ouverts et questions pour moi.
6. Prochain prompt recommandé (numéro et titre).
7. Actions qui demandent mon accord explicite (push, migration, déploiement…).

Termine par : « Étape terminée. J'attends ton accord avant de passer à la suivante. »